# Assignment 6: TDD and UML

Basic Dictionary for the Object-Oriented-Programming course

## Introduction

In this assignment, we are provided with a partially developed TicTacToe (איקס/עיגול) game. Our task is to complete the missing functionality using a Test-Driven Development (TDD) approach, meaning that we will first develop the tests before implementing the functionality itself.

TicTacToe is a simple game played on a grid of X*X, with two players: 'X' and 'O'. Each player takes turns placing their marker in an empty cell. The goal is to be the first player to fill an entire row, column, or diagonal with their marker.

The game continues until one of the players wins or the board is full, resulting in a tie. We need to track each player's wins so that, when the game ends, we can display a scoring board showing each player's scors and the overall winner (A tie is not considered a win.).

Allowed board size can rang from 3 to 10 (inclusive). The game functionality must support any board size within this allowed range.

Once a game is finished (with either a player winning or a tie), the players can choose whether to continue to another game or quit.

Below is an example of a simple run of the game (in the console):

```
Hit "y/Y" to start a new game. Or hit any other key to exit.
y
Please enter your preferred SIZE of the board (from 3 to 10. 3 -> 3x3; 4 -> 4x4; 10 -> 10x10, etc):
3
Player PLAYER-X, please enter your move. (enter a value from 1 - 9)
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+

1
Player PLAYER-O, please enter your move. (enter a value from 1 - 9)
+---+---+---+
| X |   |   |
+---+---+---+
|   |   |   |
+---+---+---+
|   |   |   |
+---+---+---+

5
Player PLAYER-X, please enter your move. (enter a value from 1 - 9)
+---+---+---+
| X |   |   |
+---+---+---+
|   | O |   |
+---+---+---+
|   |   |   |
+---+---+---+
```

```
2
Player PLAYER-O, please enter your move. (enter a value from 1 - 9)
+---+---+---+
| X | X |   |
+---+---+---+
|   | O |   |
+---+---+---+
|   |   |   |
+---+---+---+

6
Player PLAYER-X, please enter your move. (enter a value from 1 - 9)
+---+---+---+
| X | X |   |
+---+---+---+
|   | O | O |
+---+---+---+
|   |   |   |
+---+---+---+

3
PLAYER-X has won this round!
Hit "y/Y" to start a new game. Or hit any other key to exit.
n
Player PLAYER-X has won: 1 time(s).
Player PLAYER-O has won: 0 time(s).
The final winner is: PLAYER-X!!!
```

## Part – 1: Understanding the Code and Write UML

The game is partially complete, but it is missing several functionalities, such as determining the winner, allowing players to complete the game and proceed to a new one or quit, checking edge cases of input, and other details outlined in the following sections.

In this part, you will need to:

1) **Understand the current code** you have been provided
2) **Create a class diagram** representing the given classes and their relationships.
3) **Develop a sequence diagram** that illustrates the expected flow of playing the last two moves of the game, announcing the winner, and either proceeding to the next game or quitting.

**Current Progress Modifications:**
The class and sequence diagrams can be created using software such as PowerPoint or Google Slides, or they can be drawn on paper and submitted as a scan (.pdf/.png). Please submit two files: **"class_diagram"** and **"sequence_diagram"**.

## Part – 2: Developing the Tests

As mentioned, not all of the functionality has been implemented yet. Following the TDD approach, we will first implement tests for the methods listed below.

You need to create a **TicTacToeTest.java** file and implement the following tests:

1. **checkWinnerTest()**: This test should validate the **checkWinner()** method in the **TicTacToe** class, which checks whether a player ('X' or 'O') has won the game.
2. **handleWinnerTest()**: This test should ensure that the **handleWinner()** method in the **TicTacToe** class correctly manages the information needed to track the scoring.
3. **isValidPositionTest()**: This test should verify that the **isValidPosition()** method in the **Board** class does not allow a player to select an invalid cell on the board.
4. **isFullTest()**: This test should check that the **isFull()** method in the **Board** class correctly determines whether the board is full.
5. **verifyBoardSizeTest()**: This test should ensure that the **verifyBoardSize()** method in the **TicTacToe** class handles inputs correctly and does not throw any exceptions.

- A test should pass/fail using the Junit "assertion" machanism.
- For each test case, you should implement at least one test (more are advisable, though not mandatory). Keep in mind that when your exercise is evaluated, many cases will be checked.
- For additional tests not listed above that you would like to implement, you may use any name you choose (e.g., isHalfFullTest(), checkLoserTest(), etc.).

**Current Progress Modifications:** The TicTacToeTest.java class with all tests covering all possible situations

## Part – 3: Developing the Missing Functionality

After all tests are done, we should now implement the functionality of these tested methods (checkWinner, handleWinner, isValidPosition, isFull, and varifyBoardSize).
After implementing the functionality, the game should operate as described in the introduction. The tests and functionality must cover all edge cases, ensuring that the game does not crash or enter unintended scenarios. While developing the functionality, if you encounter new cases that require handling, you should add corresponding tests to verify that these scenarios are properly covered.

**Current Progress Modifications:**
Submit all the classes of the game, including your implementation:

- TicTacToe.java
- Board.java
- Player.java
- Main.java

## Part – 4: Game Simulation

In this part, you are required to implement a new type of player (and a new class), called **AutoPlayer**, which simulates a human player. This, while making no changes to the existing code, implement functionality that allows:

- Two **AutoPlayers** to play against each other.
- A human player to play against the **AutoPlayer**.

The **AutoPlayer** should remain simple and not be "smart." It should randomly select free cells, so the other player does not have to wait while the **AutoPlayer** randomly chooses an available cell. This is especially important in larger boards (e.g., a 10x10 board), where selecting the last free cell by chance can take time.

The output of a simulated game should be identical to the output of a game played by two humans. This means that the selections should be printed to the screen, and everything should occur in the same order as in a game with non-auto players. Choosing to continue to anther game or ending the game should not be automated and still excepect human input.

- Modify the Main.java class to display the following message to the user only at the very beginning of the game (**no need to ask again after a TickTacToe game ends**):
  "How many players [0-2]?"

  Depending on the user response (between 0-2) the game will be setup:
  0 -- Simulated game between two auto-players
  1 -- One human and one AutoPlayer
  2 -- Two human players.

- Use the second **TicTacToe** class constructor, which accepts two players as input, to set up different types of games (e.g., two **AutoPlayers**, or one human and one **AutoPlayer**).
- For every method implemented for **AutoPlayer**, you should add a corresponding test to verify its functionality.

**Current Progress Modifications:**

- Main.java
- AutoPlayer.java

- AutoPlayerTest.java

## Final Checks:

A ***build.xml*** file will be provided with the assignment. Before submiting, make sure that the “ant compile” and “ant test” commands run and pass successfully.

## Final Files to Submit

- class_diagram.pdf
- sequence_diagram.pdf
- TicTacToe.java
- Board.java
- Player.java
- Main.java
- AutoPlayer.java
- AutoPlayerTest.java
- build.xml (to compile your project – make sure “build success” on the server)

Good Luck!